

GAI-3101-CAP Lab Guide

Crafting Custom Agentic AI Solutions Capstone

© 2025 Ascendient, LLC

Revision 1.4.0-asc published on 2025-11-26.

No part of this book may be reproduced or used in any form or by any electronic, mechanical, or other means, currently available or developed in the future, including photocopying, recording, digital scanning, or sharing, or in any information storage or retrieval system, without permission in writing from the publisher.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

To obtain authorization for any such activities (e.g., reprint rights, translation rights), to customize this book, or for other sales inquiries, please contact:

Ascendient, LLC
1 California Street Suite 2900
San Francisco, CA 94111
<https://www.ascendientlearning.com/>

USA: 1-877-517-6540, email: getinfousa@ascendientlearning.com

Canada: 1-877-812-8887 toll free, email: getinfo@ascendientlearning.com

Table of Contents

1. Capstone: Support Agent for Bean Machine Support.....	4
1.1. Introduction.....	5
1.2. Scenario.....	5
1.2.1. SageMaker URLs	6
1.3. Resources.....	6
1.3.1. Ticketing System	6
1.3.2. Support Info	9
1.3.3. Basic LangChain OpenAPI Agent	9
1.4. Capstone Instructions	10
1.4.1. Key Requirements.....	10
1.4.2. Optional Features	10
1.4.3. Recommended Approach	11
1.4.4. Notes.....	12
1.5. Conclusion	12

Capstone: Support Agent for Bean Machine Support

LAB 1

1.1. Introduction

This capstone challenges you to create a support agent for BeanBotics Inc., a company that produces the Bean Machine, an automated coffee-making robot. The agent should read and process tickets from the ticketing system, extract key information, classify requests, and update the system accordingly.

1.2. Scenario

BeanBotics Inc. provides the Bean Machine, an automated robot arm that prepares custom coffee drinks in offices, cafes, and public spaces. The public support team receives customer requests via a shared email address. These emails contain unstructured information about issues with the Bean Machine, such as machine malfunctions, coffee quality problems, or general inquiries.

When received, these emails create support tickets in the company's ticketing system. The support team needs to quickly respond to these tickets, but the volume of incoming requests is high, and many emails contain incomplete information. The team struggles to keep up with the demand, leading to delays in responses and customer dissatisfaction.

BeanBotics has decided to build an AI agent to automate the initial processing of these support tickets. The agent will read incoming tickets, extract key information, classify the requests into categories, assign priority levels, and update the ticketing system with this information. If the email lacks necessary details, the agent will add a response asking for more information. Once the agent has this information, it will attempt to assist the customer using support documentation. If the issue cannot be resolved, the agent will escalate the ticket to a human support agent for further assistance.

1.2.1. SageMaker URLs

In this lab, we will be using a ticketing system that runs alongside your code. When running this on SageMaker, the ticketing system will be available at a URL based on your SageMaker instance. For example, if your notebook is running at

<https://cpu-instance-mljl.notebook.us-west-2.sagemaker.aws/lab/tree/basic-ticket-agent.ipynb>, the ticketing system will be available at <https://cpu-instance-mljl.notebook.us-west-2.sagemaker.aws/proxy/3000>.

However, the notebook will be able to access the ticketing system at <http://localhost:3000>. This is because the notebook and the ticketing system are running in the same environment, and the notebook can access the ticketing system using the local address.

1.3. Resources

1.3.1. Ticketing System

A simple ticketing system is provided for this lab. It is built with Node.js and provides both an OpenAPI-documented REST API and a User Interface. The system allows for ticket creation, updates, and responses.

The ticketing system supports real-time notifications via WebSocket. This allows the agent to receive updates on ticket status changes immediately, enhancing responsiveness and efficiency. The WebSocket connection will send a JSON message containing the ticket ID whenever a ticket is updated. The agent can then process this message as needed.

1.3.1.1. Key Endpoints

- [/](#) - User Interface (UI) for the ticketing system
- [/api/tickets](#) - REST API for the ticketing system
- [/api/docs](#) - OpenAPI documentation for the REST API
- [/api/docs/openapi.json](#) - OpenAPI specification in JSON format

- `/ws` - WebSocket endpoint for real-time notifications (only available if running locally)

Tip

Read through the OpenAPI documentation to understand the available endpoints and their parameters. This will help you interact with the ticketing system effectively.

1.3.1.2. Starting the Ticketing System

To start the ticketing system, perform the following steps:

1. Click the blue + button in the top-left corner of the JupyterLab interface.
2. Under "Other", select "Terminal" to open a new terminal window.
3. In the terminal, navigate to the ticketing system directory by running:

```
cd ~/SageMaker/capstone/ticketing-system
```

4. Install the required dependencies by running:

```
npm install
```

5. Start the ticketing system by running:

```
npm start
```

Note

The server will run as long as the terminal is open. You can stop it by pressing `Ctrl+C` in the terminal.

6. Open the ticketing system UI in a new browser tab.

Note

The URL for the ticketing system will be based on the URL for your notebook. For example, if your notebook is running at `https://cpu-instance-mlj1.notebook.us-west-2.sagemaker.aws/lab/tree/basic-`

`ticket-agent.ipynb`, the ticketing system will be available at `https://cpu-instance-mlj1.notebook.us-west-2.sagemaker.aws/proxy/3000`.

The ticketing system should be prepopulated with sample tickets to help you test your agent. You can view these tickets in the UI or via the REST API.

Bean Machine Ticketing System

Submit a New Ticket

Name

Email

Subject

Description

Submit Ticket

All Tickets

Subject	Status	Name	Email	Created At	Serial #	Category	Priority	Actions
Machine not grinding beans	open	Alice Smith	alice@example.com	Jun 10, 2025, 5:00 AM	BM-1001	Mechanical	High	View Delete
Coffee tastes bitter	in progress	Bob Lee	bob.lee@example.com	Jun 10, 2025, 6:30 AM	BM-2002	Quality	Medium	View Delete
Water leak detected	closed	Carol Jones	carol.j@example.com	Jun 9, 2025, 11:45 AM	BM-3003	Maintenance	High	View Delete
Machine not responding	escalated	David Kim	david.kim@example.com	Jun 8, 2025, 9:20 AM	BM-4004	Technical	High	View Delete

If the ticketing system UI doesn't load initially, try refreshing the page. If issues persist, verify the server is running in the terminal and check that you're using the correct proxy URL format. You can always ask your instructor for help!

1.3.1.3. Troubleshooting Connection Issues

WebSocket Connection Problems: WebSocket connections through the SageMaker proxy has odd behavior at times. If you encounter WebSocket errors, use HTTP polling instead (as shown in the starter notebook). This is the recommended approach for this environment.

UI Not Loading: Ensure the npm server is running in the terminal. Verify the proxy URL matches your notebook instance URL pattern. Try logging out and back into your environment if problems persist.

1.3.2. Support Info

In the `support-info` directory, you will find markdown files that provide support information for the Bean Machine. These files contain details about common issues, troubleshooting steps, and escalation procedures.

These support documents are written from the perspective of internal support staff and contain troubleshooting guides for human representatives. However, they are designed to be loaded into your agent's Retrieval-Augmented Generation (RAG) system. Your agent will use these documents to extract relevant troubleshooting steps and guidance that can be provided to customers or used to classify and resolve support tickets automatically.

The support documentation is structured to help the agent extract relevant information and guide its responses. When integrated with a RAG system (as demonstrated in the `basic-ticket-agent.ipynb` notebook), the agent can search these documents to find applicable solutions and provide appropriate assistance based on the ticket category and issue details.

1.3.3. Basic LangChain OpenAPI Agent

In the `capstone` directory, you will find a Jupyter Notebook named `basic-ticket-agent.ipynb`. This notebook serves as a basic example of how to create a LangChain agent that interacts with the Bean Machine Ticketing System's OpenAPI specification. It demonstrates how to use LangChain's OpenAPI integration to build an agent that can reason over the API and make calls to it.

It also includes examples of how to connect to a websocket endpoint and how to import the support information into a retrieval system. The notebook is designed to be a starting point for building a more advanced agent that can handle the requirements of the capstone project. You can use this notebook as a starting point for your own agent development or you can take inspiration from it to build a more advanced agent that meets the requirements of the capstone project.

1.4. Capstone Instructions

This capstone is designed to be open-ended, allowing you to design and implement an agent that meets the requirements of the BeanBotics support team. The following sections outline the key features, information to extract, and the ticketing system integration.

While we recommend using the provided `basic-ticket-agent.ipynb` as a starting point, you are encouraged to explore and implement your own solutions. You can use any libraries or frameworks that you find suitable for building the agent, as long as it meets the requirements outlined below.

1.4.1. Key Requirements

The agent must be able to:

- Read tickets from the ticketing system (via the REST API)
- Extract key information from the ticket content
- Classify the request into predefined categories (e.g., cleaning/maintenance, coffee quality, software updates, feature requests)
- Assign a priority level based on urgency indicators
- Update the ticketing system with the extracted information

Note

The goal is to build an automated ticket classification and processing system, not an interactive chatbot. Your agent should analyze incoming support tickets, categorize them, extract relevant information, and update the ticketing system accordingly. The agent may also respond to tickets asking for additional information if needed, or provide assistance based on the support documentation.

1.4.2. Optional Features

The following features are optional but can enhance the agent's capabilities:

- Ask the customer for more information if key details are missing

- Use the support documentation to assist in resolving issues
- Escalate tickets to human support agents if the issue cannot be resolved
- Integrate with WebSocket for real-time updates on ticket status changes

1.4.3. Recommended Approach

We recommend the following approach to building your agent:

1. **Set Up the Environment:** Ensure you have the ticketing system running and accessible via the provided URL. Familiarize yourself with the OpenAPI documentation and the support information available in the `support-info` directory.
2. **Review the Basic Agent:** Open the `basic-ticket-agent.ipynb` notebook and review the code. Understand how it interacts with the ticketing system and how it uses LangChain's OpenAPI integration.
3. **Make a Plan for Your Agent:** Outline the key features you want to implement in your agent. Consider how you will extract information, classify requests, and handle missing details.
4. **Implement Key Information Extraction:** Use LangChain's tools to extract key information from the ticket content. This may include identifying the issue type, urgency, and any specific details mentioned in the ticket. You may either use the OpenAPI integration or implement your own custom tools to wrap the ticketing system's API.

To start, use a hard-coded ticket ID to test your agent.

5. **Respond to Missing Information:** Implement logic to check if the ticket contains all necessary information. If not, use the agent to generate a response asking for more details.
6. **Implement WebSocket Integration:** If you choose to implement real-time updates, set up a WebSocket connection to the ticketing system. This will allow your agent to receive notifications about ticket status changes immediately.

7. **Add Support Documentation Integration:** Use the support information in the `support-info` directory to assist in resolving issues. Implement logic to search for relevant documentation based on the extracted ticket information. The notebook demonstrates how to load these documents into a RAG system so your agent can retrieve and use the troubleshooting guidance when processing tickets.

1.4.4. Notes

This capstone is designed to be open-ended. You may not complete all the features listed above, but you should aim to implement a functional agent that can read tickets, extract key information, and update the ticketing system.

You are encouraged to explore and experiment with different approaches to building your agent. The goal is to create a support agent that can effectively assist the BeanBotics support team in managing customer requests.

If you have time, consider implementing additional features and enhancements to your agent.

Note

This capstone is operates in a simulated environment. The ticketing system is basic and the support information is limited. Feel free to extend the support documentation with additional information or additional features to the ticketing system if you want to test your agent with more complex scenarios.

1.5. Conclusion

In this capstone, you have the opportunity to design and implement a support agent for BeanBotics Inc. The agent will help automate the processing of support tickets, improving response times and customer satisfaction. By leveraging LangChain's capabilities and integrating with the ticketing system, you can create a powerful tool that enhances the support team's efficiency.

